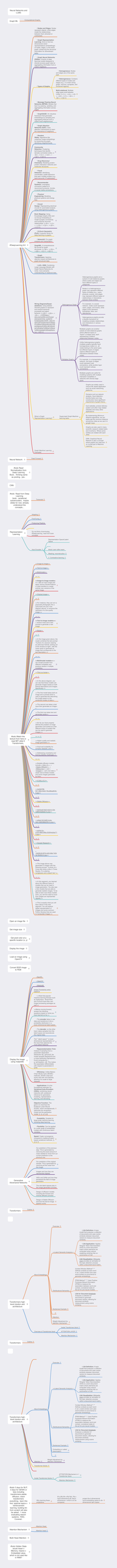

Neural Networks and LLMS

- Graph ML
  - Computational Graphs
- #DeepLearning 6.6
  - • **Nodes and Edges:** Nodes represent objects, while edges model the relationships between them (directed or undirected).
  - • **Graph Representation Learning:** Aims to find low-dimensional vector representations (embeddings) of nodes, edges, or the entire graph, preserving the structural information for downstream tasks.
  - • **Graph Neural Networks (GNNs):** A family of deep learning models designed to directly process graph data by aggregating information from a node's neighbors.
  - • **Types of Graphs:**
    - • **Homogeneous:** Nodes and edges are of the same type.
    - • **Heterogeneous:** Contains different types of nodes and edges (e.g., in a brain wiring graph: neurons, synapses, and functional regions).
    - **Multi-relational:** Multiple edge types exist between nodes. [++1++, ++4++, ++5++, ++6++, ++7++, ++8++, ++9++, ++10++]
  - • **Message Passing Neural Networks (MPNN):** Models that learn by passing, updating, and aggregating information across edges.
  - • **GraphSAGE:** An inductive framework that generates embeddings by sampling and aggregating features from a node's local neighborhood.
  - • **Graph Attention Networks (GAT):** Uses attention mechanisms to learn the importance of neighbors.
  - • **Random Walks:** Algorithms like node2vec create embeddings by traversing the graph, identifying neighborhoods.
  - **Community Detection:** Clustering techniques that identify densely connected nodes. [++9++, ++11++, ++12++, ++13++, ++14++]
  - • **Drug Discovery/ Chemistry:** Modeling protein-protein interaction networks and molecular structures.
  - • **Fraud Detection:** Identifying coordinated, subtle behaviors that span multiple entities (e.g., bank accounts, IP addresses).
  - • **Recommender Systems:** Leveraging connection patterns to recommend products, movies, or social media connections.
  - • **Physics/ Engineering:** Modeling physical systems and energy flow.
  - • **Circuit Design:** Representing electrical circuits for automated analysis using heterogeneous graphs.
  - **Brain Mapping:** Using Knowledge Graphs (KG) and Lie Group Embedding to model synaptic connectivity and functional neuronal relationships. [++6++, ++11++, ++13++, ++15++, ++16++, ++17++]
  - • **PyTorch Geometric (PyG):** A popular library for deep learning on graphs.
  - • **NetworkX:** For graph analysis and manipulation.
  - **GraphML:** A comprehensive file format for graph structures. [++18++, ++19++, ++20++, ++21++, ++22++]
  - • **Graph Transformers:** Applying attention-based architectures to graph-structured data.
  - • **LLM + GNN:** Combining Large Language Models with Graph Neural Networks for richer, more semantic understanding of relational data.
  - **Wiring Diagrams/Quasi-skeletons:** Using labeled directed graphs to represent and analyze temporal processes and agent behavior. [++23++, ++24++, ++25++, ++[26]](https://docs.techsoft3d.com/hoops/ai/programming_guide/ml-fundamentals.html#:~:text=Graph%20Transformers%20apply%20the%20transformer%20architecture%20(a.all%20nodes%20rather%20than%20just%20immediate%20neighbors.)++, ++27++]
    - Heterogenous Graph
      - Heterogeneous graphs are a subset of multi-relational graphs where nodes and edges can have different types or categories.
      - Nodes in a heterogeneous graph may represent different types of entities (e.g., users, products, events), and edges may represent different types of relationships or interactions between these entities.
      - For example, in a social network, nodes could represent users, pages, and events, while edges could represent friendships, likes, and attendances.
      - Heterogeneous graphs provide a flexible framework for modeling complex relationships and interactions in various domains.
    - Multiplex Graphs
      - Multiplex graphs are another subset of multi-relational graphs where different layers or "layers" of edges exist, each representing a distinct relationship between nodes.
      - Unlike heterogeneous graphs, multiplex graphs typically have homogeneous nodes (i.e., all nodes belong to the same type), but different layers of edges capture different types of interactions between these nodes.
      - For example, in a transportation network, one layer of edges could represent road connections, while another layer could represent railway connections.
      - Multiplex graphs are useful for modeling systems with multiple interaction modalities or networks with diverse edge types.
  - What is Graph Representation Learning?
    - Supervised Graph Machine learning tasks
      - Graphs are widely used in various ML based application such as social networking platforms.
      - Domains such as network analysis, fraud detection, bioinformatics and map construction have significant requirement of graph theory.
      - food websites, product delivery system and other map related websites and many other applications.
      - By implementing Minimum distance algorithms, we can easily get the minimum distance among two cities as we use it in google maps.
      - Graphs are also used in many scientific research related tasks and inventions. Where some entities are related with each other.
      - GNN, Graphical Neural Network is also a concept where graphs are used and on crucial part of Machine Learning.
  - Graph Machine Learning Examples
- Neural Network
  - Feed Forward
- #todo Read Normalization from Deep Learning Book... thinking same as pooling.. yes..
- CNN
- #todo Read from Deep Learning book.. variational autoencoders.. maybe slides for now. already understood the concepts...
  - Transcript
- Representation Learning
  - Adapting
  - FineTuning
  - Finetuning Pipeline
  - Do not think of terminolog, #DeepLearning read from book concepts
  - Auto Encoder
    - Representation Space/Latent Space
    - #todo Learn after exam..
    - Masking: reconstruction
    - 2. Contrastive learning
- #todo Watch the lecture from here or read MIT slide 9 Transformers,
  - ++Image-to-image++
  - ++Text-to-image++
  - ++Multimodal++
  - ++ ++
  - **++Image-to-image models++** take an input image and apply various transformations or style transfers to create entirely new versions of the same image.
  - ++Original image++
  - ++
  - ++For instance, they can turn a grayscale photograph into a colourized one (as in the diagram above), or enhance the resolution of a low-quality image.++
  - ++ ++
  - **++Text-to-image models++** receive noise and a text prompt to generate a new image.
  - ++Noise++
  - ++
  - ++In the image given above, the model combines the text prompt "A photo of a raccoon wearing an astronaut helmet, looking out of the window at night", with the noise vector to generate an image that captures the text description.++
  - ++ ++
  - **++Multimodal models++** can accept prompts from different modalities and produce results in multiple modalities.
  - ++This is a horse++
  - ++
  - ++In the above diagram, you can see that these models can generate images based on both textual descriptions and images. Specifically:++
  - ++The first model takes both an image and a prompt which it uses to generate text describing the image based on the combined modes of data++
  - ++The second one takes a text input then generates an image++
  - ++The third one takes text and generates audio++
  - ++ ++
  - ++So far you have properly defined the problem of image generation and looked at a few different kinds of models that can be used to generate images.++
  - ++ ++
  - ++Higher quality and realistic image generation.++
  - ++Improved scalability for complex datasets, and++
  - ++Addressing uncertainty and training stability challenges.++
  - ++ ++
  - ++Notable diffusion models include++ DALL-E++, ++Stable Diffusion++, ++Midjourney++, and ++Imagen++. Let's now take a closer look at these models and the images generated by them.
  - ++5 DALL-E 2++
  - ++
  - ++4e4fd798-ff57-48dc-9cb2-76cc86ea854b-A.jpg++
  - ++
  - ++Stable Diffusion++
  - ++faf42ee6-fbd0-4bd4-b126-cd8654da5de5-A.png++
  - ++
  - ++afdad146-9d63-4c4e-bf0c-0d85388d3f4-A.png++
  - ++
  - ++ebf3d12c-d720-4863-9f9a-2535f4a3da77-A.jpg++
  - ++
  - ++Google Research++
  - ++
  - ++0b0ffc59-8f78-4d55-86e7-604b1f84943-A.jpg++
  - ++
  - ++The image above was generated in Imagen with the following prompt: "A photo of a Corgi dog riding a bike in Times Square. It is wearing sunglasses and a beach hat."++
  - ++ ++
  - ++In this segment, you learned about the different types of models that can be used in image generation. You also saw a few models that are able to generate realistic images. To be able to learn how these models work, you will first need to know how images are represented digitally.++
  - ++This is exactly what you will learn about in the next segment. You will explore digital representation of images and some common Python libraries that can be used to manipulate images.++
- Open an image file
- Get image size
- Get pixel color at a specific location (x, y)
- Display the image
- Load an image using OpenCV
- Convert BGR image to RGB
- Display the image using matplotlib
  - • *NumPy*
  - • *OpenCV*
  - • *Matplotlib*
  - Image Processing using OpenCV
  - • ++Note that popular machine learning libraries such as *Scikit-learn*, *TensorFlow*, and *PyTorch* have their own image processing packages as well.++
  - ++Before moving forward, answer the following questions based on what you have learned so far.++
  - • **The encoder** takes in raw data and transforms it into a probability distribution in a lower-dimensional latent space.
  - • **The decoder**, on the other hand, takes samples from the latent space and reconstructs the original data.
  - The** latent space** is lower dimensional representation of the input that captures the most salient features
  - • **Reparameterization Trick:** Instead of sampling from a parameter-dependent distribution $q_\phi(z|x)$, the model samples $\epsilon$ from a noise distribution $p(\epsilon)$ and computes $z = g_\phi(\epsilon, x)$. This makes the expectation differentiable with respect to parameters $\phi$.
  - • **Efficiency:** Unlike Markov Chain Monte Carlo (MCMC) methods, SGVB is fast and works well with mini-batches, allowing it to scale to large datasets.
  - • **Applications:** It is the foundational estimator for **Variational Auto-Encoders (VAEs)**, combining neural networks with variational inference for generative modeling, representation learning, and denoising.
  - **Objective Function:** The goal is to maximize the Evidence Lower Bound (ELBO), which simultaneously optimizes the recognition model and the generative model. [1, 3, 4, 5, 6]
  - • **Scalability:** Suitable for large-scale machine learning, including deep learning.
  - • **Flexibility:** Can be applied to a wide range of probabilistic models with continuous latent variables.
  - **Speed:** Faster convergence compared to traditional batch-based variational methods. [1, 2, 6, 7]
- Generative Adversarial Networks
  - Our prediction of the previous step: This is generated by removing the exact amount of noise needed to move from our current prediction to the previous one.
  - Our prediction of the original sample: This is generated by removing all the noise from the image.
  - Images and how they are represented digitally
  - VAEs and GANs and how they pioneered the field of image generation
  - The role latent spaces play in the image generation process
  - Design of diffusion models including the forward and reverse diffusion process
  - Design of Stable Diffusion and how the text-to-image pipeline works
- Transformers
  - Outline
- Transformers high-level intuition and architecture
  - Word Embeddings
    - Overview
    - ++Latent Semantic Analysis++
      - • **LSA Definition:** A topic model that analyzes a collection of documents and uses cosine similarity between document vectors to measure document similarity.
      - • **LSA Application:** Applies Singular Value Decomposition (SVD) to a term-document matrix where elements are computed using various weighting schemes like co-occurrence or tf-idf.
      - **LSA Visualization:** Wikipedia page provides an animation of the topic detection process within a document-word matrix.
    - Distributional Semantics
      - Context Window Method**:** Defines contexts as each word in an L-sized window and uses word-context co-occurrence to generate embeddings.
      - PPMI Method**:** Uses Positive Pointwise Mutual Information (PPMI) to measure the association between word and context, considered state-of-the-art for pre-neural models.
      - **LSA for Document Analysis:** Analyzes a collection of documents to generate document vectors, allowing for document similarity measurement using cosine similarity.
    - Sentences Example
    - Attention
    - Weight Adjustment by attention mechanism
  - Overview of Transformer block
    - Inside Transformer block
    - ATTENTION LAYER
    - Attention Mechanism
- Transformers
  - Outline
- Transformers high-level intuition and architecture
  - Word Embeddings
    - Overview
    - ++Latent Semantic Analysis++
      - • **LSA Definition:** A topic model that analyzes a collection of documents and uses cosine similarity between document vectors to measure document similarity.
      - • **LSA Application:** Applies Singular Value Decomposition (SVD) to a term-document matrix where elements are computed using various weighting schemes like co-occurrence or tf-idf.
      - **LSA Visualization:** Wikipedia page provides an animation of the topic detection process within a document-word matrix.
    - Distributional Semantics
      - Context Window Method**:** Defines contexts as each word in an L-sized window and uses word-context co-occurrence to generate embeddings.
      - PPMI Method**:** Uses Positive Pointwise Mutual Information (PPMI) to measure the association between word and context, considered state-of-the-art for pre-neural models.
      - **LSA for Document Analysis:** Analyzes a collection of documents to generate document vectors, allowing for document similarity measurement using cosine similarity.
    - Sentences Example
    - Embedding or called tokenization
  - Attention
    - Weight Adjustment by attention mechanism
  - Transformer blocks
  - Inside Transformer blocks
    - ATTENTION Mechanism in 1 Transformer block
    - Attention Mechanism
- #todo 2 days for NLP, 3 days for GENAI or less including transformers,stable diffusion, vision transformers everything.. learn this first, optional topics-> reinforcement learning, building llm from scratch will also be asked.. 1 more evaluating search systems.. RAG.. Covered..
  - Why learning these projections
    - W q,Wk,Wv->R(d*dk), Rd-> Input dimensions,dk->key's dimensions-> which is to be compared..
      - project the d-dimensional word-embedding space to dk-dimensional (qkv) space
- Attention Mechanism
  - Attention Head
  - Attention head
- Multi Head Attention
- #todo Hidden State would mean-> Memory, means-> Embedded value, which is not used like in RNN?